TECNOLÓGICO NACIONAL DE MÉXICO

TECNOLÓGICO NACIONAL DE MÉXICO

INSTITUTO TECNOLÓGICO DE ZACATEPEC

Ingeniería en Sistemas Computacionales

Materia:

Fundamentos de Ciencia de Datos

Grupo:

XB

Profesora:

Ramos Palencia Celia

Unidad 2. Toma de Decisiones e Inferencia Estadística

REPORTE TÉCNICO

Actividad Integradora: Evaluación Práctica de la Unidad 2

Inferencia Estadística y Toma de Decisiones en Ingeniería

Caso A: Arquitectura de Software (Monolito vs. Microservicios)

Alumnos:

Hermosa Praxedis Carlos Rafael

Muñoz García Jorge

Suárez Degante Luis Ángel

Morantes Pereida Denzel Yahir

Fecha de entrega:

18 de octubre de 2026

Zacatepec de Hidalgo, Morelos — Octubre de 2026

Contenido

1. Introducción **3**
2. Objetivo **3**
3. Selección y justificación del caso **3**
4. Metodología **4**
4.1 Diseño experimental 4
4.2 Obtención del dataset 4
4.3 Herramientas 4
5. Bloque 1 — Estadística descriptiva y limpieza **5**
5.1 Carga y verificación 5
5.2 Remoción de atípicos (criterio 1.5 × IQR) 5
5.3 Medidas descriptivas 5
5.4 Interpretación 6
6. Bloque 2 — Distribuciones de muestreo y Teorema Central del Límite **7**
Justificación de la convergencia a la normalidad 9
7. Bloque 3 — Estimación por intervalos de confianza **9**
7.1 IC para la media poblacional general μ 9
7.2 IC del 95 % para la diferencia de medias μ1 − μ2 10
7.3 Interpretación de 0 ∉ IC 10
8. Bloque 4 — Teoría de decisión estadística (prueba de hipótesis) **11**
8.1 Planteamiento de hipótesis 11
8.2 Matriz de decisión y costo de los errores 11
8.3 Nivel de significancia 11
8.4 Ejecución de la prueba paramétrica 11
8.5 Riesgo de Error Tipo II (β) y potencia 12
8.6 Veredicto 12
9. Recomendación técnica de ingeniería **12**
10. Limitaciones y trabajo futuro **13**
11. Conclusiones **13**
Anexo A. Estructura del repositorio **14**

Repositorio de Github: https://github.com/vesseldot/fcd-u2-caso-a/tree/main
Cuaderno de Colab:
https://colab.research.google.com/drive/1ft65UVF2KJg6lyPsBh8VBJoYIBxaRG4J?usp=sharing

1. Introducción

La migración de aplicaciones monolíticas hacia arquitecturas de microservicios es una de las decisiones de diseño más frecuentes en la ingeniería de software actual. Los microservicios ofrecen escalado independiente, despliegues aislados y equipos autónomos, pero a cambio convierten las llamadas a función en memoria en llamadas remotas por red. Cada salto entre servicios implica abrir o reutilizar una conexión TCP, serializar y deserializar mensajes y atravesar la pila HTTP, lo que podría incrementar la latencia percibida por el usuario.

Una decisión de arquitectura de este tipo no debe tomarse por intuición. En este reporte se aplica el ciclo completo de la inferencia estadística sobre telemetría real de latencia, obtenida mediante una prueba de carga controlada, para sustentar formalmente una recomendación técnica.

2. Objetivo

Determinar, con un nivel de significancia $\alpha = 0.05$, si la sobrecarga de red introducida por una arquitectura de microservicios incrementa la latencia media de respuesta HTTP respecto a un monolito que ejecuta exactamente la misma lógica de negocio, empleando Python (numpy, pandas, matplotlib, seaborn y scipy).

Elemento	Definición
Variable cuantitativa (Y)	**latencia_ms**: latencia extremo a extremo de una petición HTTP, en milisegundos
Variable cualitativa (X)	**grupo**: tipo de arquitectura (Monolito / Microservicios)
Unidad de observación	Una petición GET /pedido (usuario + inventario + precio)
Tamaño muestral	300 observaciones por grupo (N = 600); 579 tras la depuración

3. Selección y justificación del caso

Se evaluaron los cuatro casos disponibles con cinco criterios: posibilidad de obtener datos reales, control experimental, claridad de la hipótesis, reproducibilidad y relevancia para la toma de decisiones en ingeniería.

Criterio	A · Monolito vs. Microservicios	B · PostgreSQL vs. MongoDB	C · Con CI vs. Sin CI	D · HTTP vs. MQTT
Datos reales obtenibles	Sí: prueba de carga local	Requiere instalar y configurar 2 motores	Sí (API de GitHub), pero clasificar «con/sin CI» es ambiguo	Requiere hardware IoT o emulación
Control experimental	Alto: misma lógica, solo cambia la arquitectura	Medio: esquemas y consultas difieren	Bajo: muchas variables de confusión	Medio
Hipótesis direccional clara	Sí ($\mu_{micro} > \mu_{mono}$)	No (bilateral)	Sí	Sí
Reproducibilidad	Total (scripts incluidos)	Media	Baja (los datos cambian con el tiempo)	Baja
Evaluación	**Óptimo**	Viable	Riesgoso	Riesgoso

Se eligió el Caso A porque es el único que permite generar datos reales en un experimento controlado, donde la lógica de negocio es idéntica y la única diferencia sistemática entre grupos es la comunicación por red. Esto fortalece la validez de la conclusión: si se encuentra una diferencia, puede atribuirse causalmente a la arquitectura.

4. Metodología

4.1 Diseño experimental

Se implementaron dos arquitecturas en Python (biblioteca estándar, módulo http.server), descritas en src/prueba_carga.py. Ambas resuelven la operación “consultar pedido”, que valida al usuario, calcula disponibilidad en almacenes y calcula el precio con descuento:

Arquitectura	Componentes	Comunicación interna
Monolito	1 proceso (puerto 8000) con los 3 módulos	Llamadas a función en memoria
Microservicios	API Gateway (8100) + usuarios (8101) + inventario (8102) + precios (8103); cada uno en su propio proceso del SO	3 peticiones HTTP/JSON secuenciales desde el gateway, con una conexión TCP nueva por llamada

4.2 Obtención del dataset

El script *src/prueba_carga.py* levanta los cinco procesos, ejecuta 40 rondas de calentamiento y envía 300 peticiones por arquitectura en orden aleatorio intercalado (semilla 42), con identificadores de usuario y producto aleatorios y un tiempo de reflexión uniforme de 0 a 10 ms entre peticiones. La latencia se mide en el cliente con *time.perf_counter()*. La aleatorización garantiza que el ruido del sistema operativo afecte por igual a ambos grupos.

```
plan = ["Monolito"] * N + ["Microservicios"] * N
rng.shuffle(plan)                    # orden aleatorio intercalado
for i, grupo in enumerate(plan, start=1):
    uid, pid = rng.randint(1, 2000), rng.randint(1, 500)
    t0 = time.perf_counter()
    urllib.request.urlopen(f"{ENDPOINTS[grupo]}?uid={uid}&pid={pid}").read()
    latencia_ms = (time.perf_counter() - t0) * 1000
```

El resultado se almacenó en *data/dataset_caso_a_raw.csv* (600 registros, 6 columnas: id_peticion, timestamp, grupo, uid, pid y latencia_ms). El cuaderno analiza este archivo, por lo que todos los resultados son reproducibles.

4.3 Herramientas

Python 3.13, numpy 2.5, pandas 3.0, scipy 1.18, matplotlib y seaborn 0.13. El cuaderno *notebooks/caso_a_monolito_vs_microservicios.ipynb* encapsula la lógica estadística en funciones reutilizables (*filtrar_iqr*, *resumen_descriptivo*, *simular_tcl*, *ic_media*, *ic_diferencia*) y está documentado celda a celda.

5. Bloque 1 — Estadística descriptiva y limpieza

5.1 Carga y verificación

El DataFrame contiene 600 filas: *latencia_ms* es float64, *grupo* se convirtió a tipo categórico, *timestamp* a datetime64 y los identificadores son enteros. No se encontraron valores ausentes ni registros duplicados, por lo que no fue necesaria la imputación. Cada grupo tiene 300 observaciones.

5.2 Remoción de atípicos (criterio 1.5 × IQR)

Los límites LI = $Q_1 - 1.5 \cdot IQR$ y LS = $Q_3 + 1.5 \cdot IQR$ se calcularon por grupo, ya que ambas distribuciones están en escalas distintas: un límite global eliminaría datos legítimos de Microservicios.

Grupo	Q1	Q3	LI	LS	n original	Atípicos	n depurado
Monolito	1.2006	1.4599	0.8117	1.8488	300	11 (3.7 %)	289
Microservicios	3.5805	4.2970	2.5057	5.3717	300	10 (3.3 %)	290

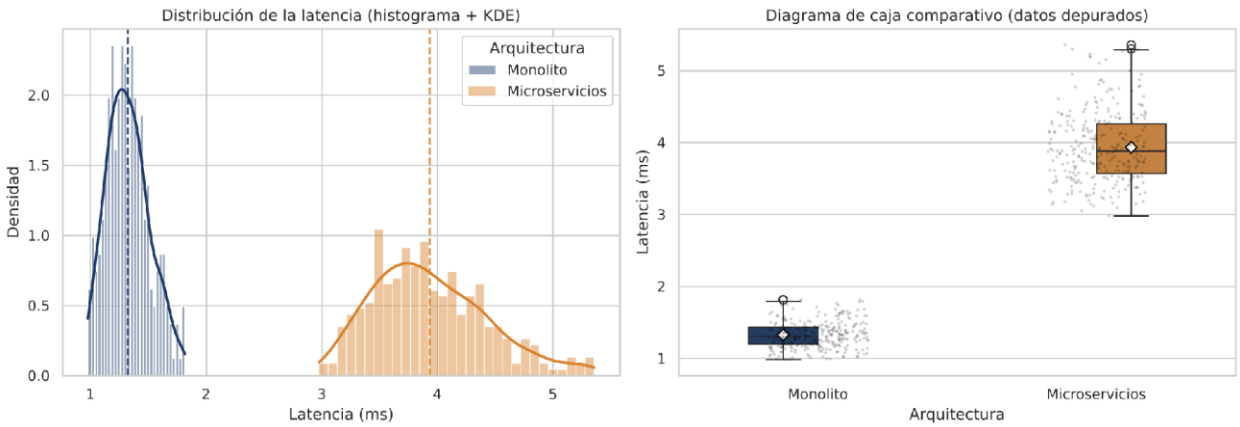
Límites de Tukey y observaciones removidas (ms).

Todos los atípicos estaban en el extremo superior: son picos esporádicos causados por el planificador del SO, la recolección de basura de Python o contención momentánea de CPU. Tras la limpieza quedan 290 y 289 observaciones (N = 579), por encima del mínimo de 200 por grupo. El dataset depurado se guardó en *data/dataset_caso_a_limpio.csv*.

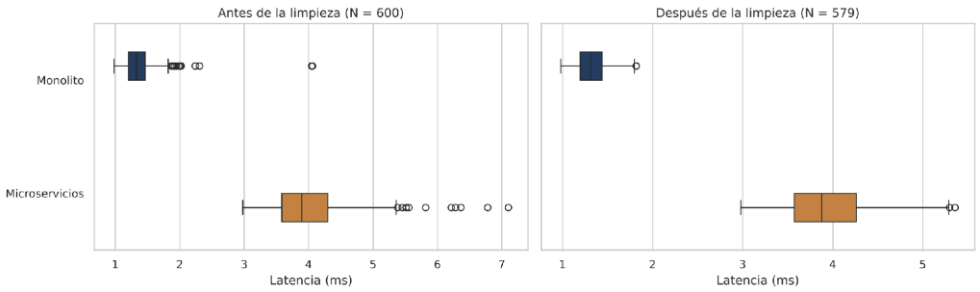
5.3 Medidas descriptivas

Grupo	n	Media $\bar{x}$	Mediana	s	s^2	Asimetría	Curtosis*
Monolito (antes)	300	1.3667	1.3239	0.3091	0.0955	4.687	36.887
Monolito (después)	289	1.3278	1.3093	0.1837	0.0337	0.408	-0.291
Microservicios (antes)	300	4.0074	3.8939	0.6234	0.3887	1.540	3.919
Microservicios (después)	290	3.9372	3.8789	0.4925	0.2425	0.606	0.026

Estadísticos muestrales de latencia (ms) antes y después de la limpieza.



Histogramas con curva KDE (izq.; líneas punteadas = medias) y diagramas de caja con datos individuales (der.).



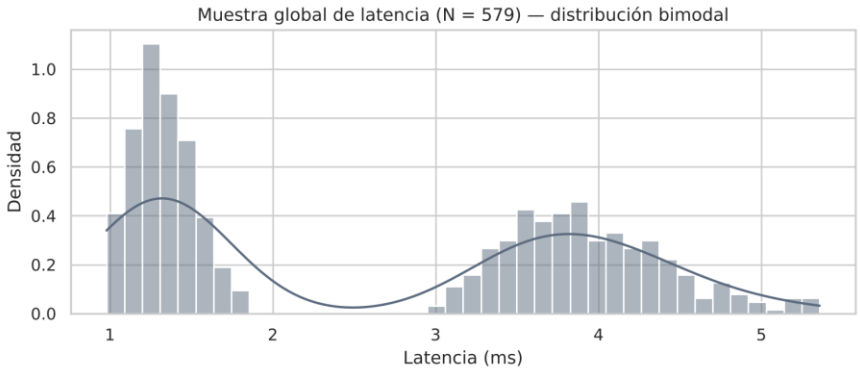
Efecto de la depuración por IQR sobre la dispersión.

5.4 Interpretación

- **Tendencia central.** El Monolito responde en promedio en 1.328 ms y Microservicios en 3.937 ms, es decir, **2.97 veces más** (+196.5 %). Las cajas no se traslapan: el mínimo de Microservicios (2.978 ms) supera al máximo del Monolito (1.818 ms).
- **Dispersión.** Microservicios también es más variable: s = 0.492 ms frente a 0.184 ms, y su varianza es 7.2 veces mayor. Cada salto HTTP aporta su propia variabilidad y las varianzas se suman; esto anticipa que no se cumplirá la homogeneidad de varianzas.
- **Sesgo.** Ambas distribuciones tienen asimetría positiva (cola a la derecha: 0.606 y 0.408), patrón típico de latencias: existe un piso físico, pero pueden ocurrir picos. La media queda ligeramente por encima de la mediana.
- **Curtosis.** Antes de la limpieza el Monolito tenía un exceso de curtosis de 36.9 (colas muy pesadas por los picos). Después, los valores son cercanos a 0 (0.026 y -0.291): la depuración eliminó las colas extremas sin deformar el cuerpo de la distribución.

6. Bloque 2 — Distribuciones de muestreo y Teorema Central del Límite

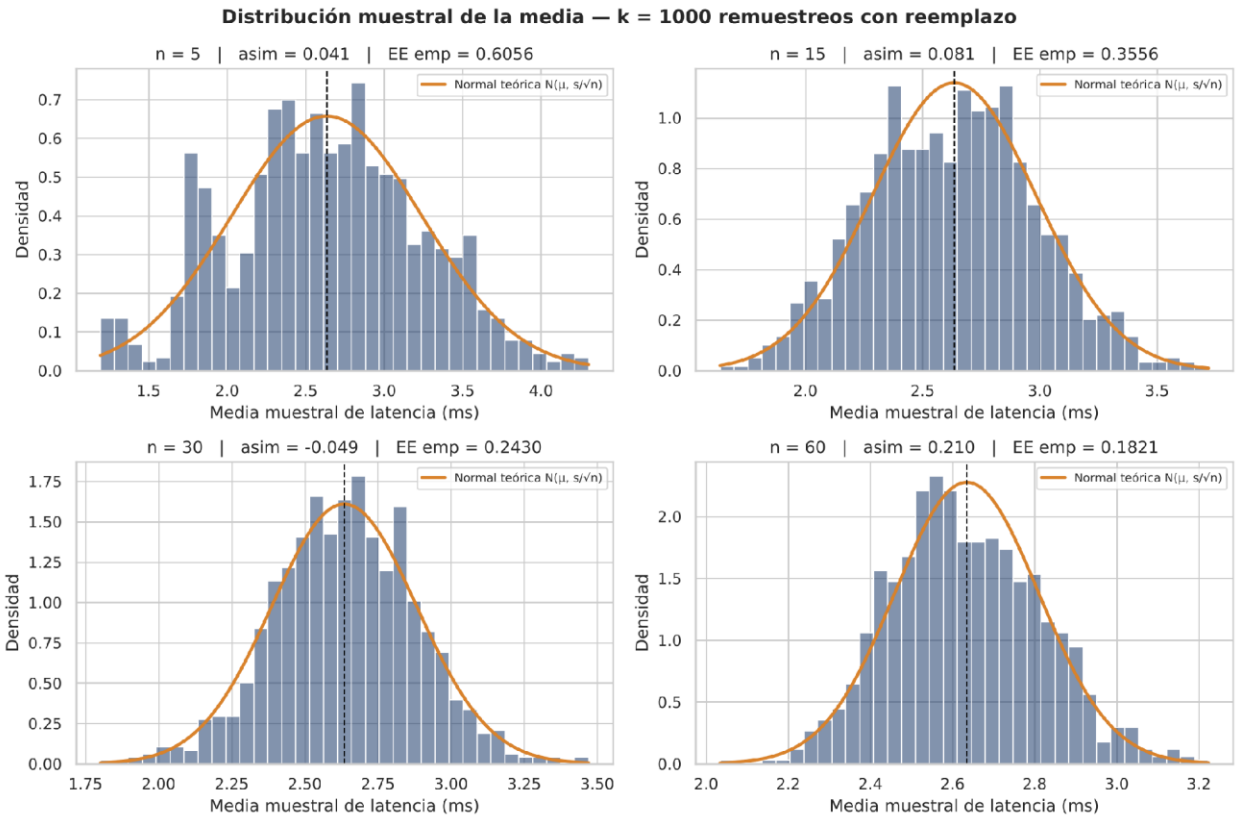
Se aisló la latencia de todas las peticiones depuradas (N = 579, media = 2.6348 ms, s = 1.3577 ms). Esta población es deliberadamente **no normal**: al mezclar las dos arquitecturas resulta **bimodal** (Shapiro-Wilk $p = 4.93 \times 10^{-24}$), lo que la convierte en una prueba exigente del TCL.



Distribución de la muestra global: bimodal y no normal.

Se programó un remuestreo con reemplazo de k = 1000 muestras para cada tamaño $n \in \{5, 15, 30, 60\}$:

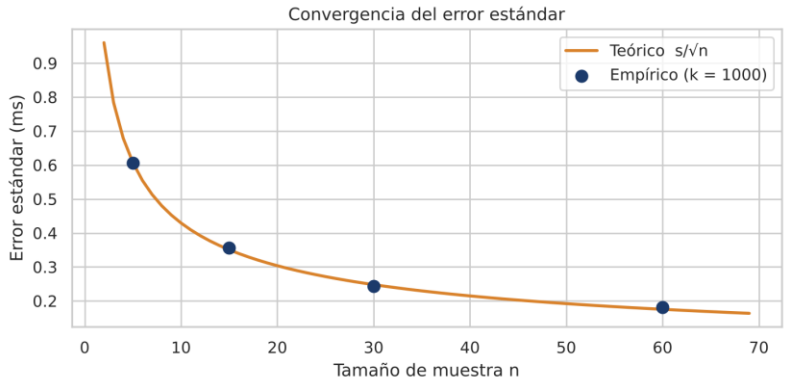
```
def simular_tcl(datos, tamanos=(5, 15, 30, 60), k=1000, rng=rng):
    return {n: rng.choice(datos, size=(k, n), replace=True).mean(axis=1)
            for n in tamanos}
medias = simular_tcl(global_lat, TAMANOS, K, rng=np.random.default_rng(42))
```



Distribución de las 1000 medias muestrales por tamaño n, con la curva normal teórica N(μ, s/√n).

n	Media de medias	EE empírico	EE teórico s/√n	Error relativo	Asimetría	Curtosis*
5	2.6288	0.6056	0.6072	0.26 %	0.041	-0.366
15	2.6243	0.3556	0.3505	1.44 %	0.081	-0.359
30	2.6387	0.2430	0.2479	1.97 %	-0.049	0.262
60	2.6316	0.1821	0.1753	3.90 %	0.210	-0.163

Error estándar empírico vs. teórico.



El EE empírico sigue la curva teórica s/√n.

Justificación de la convergencia a la normalidad

- **Centro:** en los cuatro tamaños la media de las medias coincide con la media global (2.635 ms), lo que confirma que x̄ es un estimador insesgado de μ.

- **Dispersión:** el EE empírico difiere del teórico en menos de 4 % y decrece proporcionalmente a $1/\sqrt{n}$: al cuadruplicar n (15 → 60) el EE pasa de 0.3556 a 0.1821 ms, aproximadamente la mitad. Las pequeñas discrepancias se deben a que k = 1000 es finito.
- **Forma:** aunque la población es bimodal, desde n = 5 la distribución de medias ya es unimodal y simétrica; en todos los tamaños la asimetría y el exceso de curtosis quedan en $|\cdot| < 0.4$, propios de una normal, y el ajuste a la curva teórica mejora conforme n crece. La prueba de Shapiro-Wilk sobre 1000 medias es tan sensible que detecta desviaciones mínimas, por lo que la evaluación se basó en la forma, la asimetría y la curtosis.

Conclusión del Bloque 2. Se verifica empíricamente el Teorema Central del Límite: $\bar{x} \sim N(\mu, \sigma/\sqrt{n})$ aproximadamente, sin importar la forma de la población. Esto justifica el uso de los intervalos t y de la prueba t de los bloques siguientes, aun cuando las latencias individuales presentan sesgo.

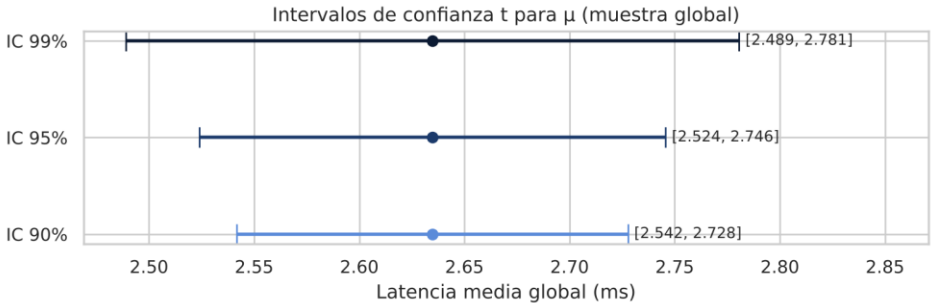
7. Bloque 3 — Estimación por intervalos de confianza

7.1 IC para la media poblacional general μ

Selección de la distribución. La desviación estándar poblacional σ es desconocida y se estima con s; por lo tanto, el estadístico $(\bar{x} - \mu)/(s/\sqrt{n})$ sigue una distribución **t de Student con n − 1 grados de libertad**, y el intervalo es $\bar{x} \pm t_{\alpha/2,\, n-1} \cdot s/\sqrt{n}$. Con n = 579 los grados de libertad son tantos que t ≈ Z, por eso se reporta también el intervalo Z como comparación: las diferencias aparecen en la cuarta cifra decimal. La t es la elección formalmente correcta y, además, conservadora.

Confianza	t crítico	IC t (ms)	z crítico	IC Z (ms)	Margen de error
90 %	1.6475	[2.5418, 2.7277]	1.6449	[2.5419, 2.7276]	± 0.0930
95 %	1.9641	[2.5239, 2.7456]	1.9600	[2.5242, 2.7453]	± 0.1108
99 %	2.5844	[2.4889, 2.7806]	2.5758	[2.4894, 2.7801]	± 0.1458

Intervalos de confianza para μ (N = 579, $\bar{x}$ = 2.6348 ms, EE = 0.0564 ms).



A mayor nivel de confianza, intervalo más ancho.

A mayor confianza el intervalo se ensancha (t crítico de 1.65 a 2.58): más confianza implica menos precisión. Sin embargo, esta media global mezcla dos poblaciones distintas y tiene poca utilidad práctica; el intervalo relevante para la decisión es el de la **diferencia** de medias.

7.2 IC del 95 % para la diferencia de medias $\mu_1 - \mu_2$

Se define μ_1 = latencia media de Microservicios y μ_2 = latencia media del Monolito, de modo que una diferencia positiva representa la penalización de los microservicios.

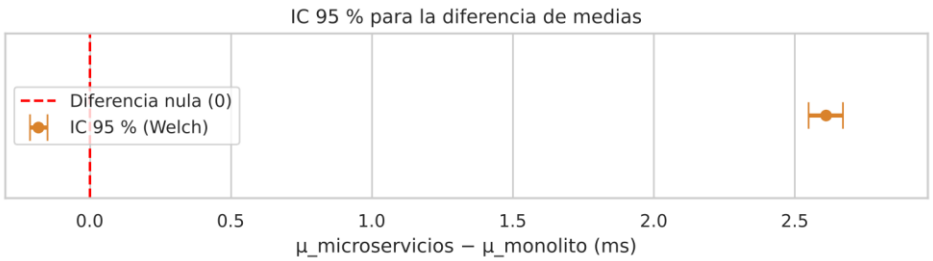
Paso 1. Prueba de Levene (H_0: $\sigma_1^2 = \sigma_2^2$; $\alpha = 0.05$; centro = mediana, variante de Brown-Forsythe, robusta ante el sesgo): $W = 165.33$, $p = 1.89 \times 10^{-33}$. Como $p < 0.05$ se **rechaza la homogeneidad de varianzas**, en concordancia con la razón de varianzas de 7.2 observada en el Bloque 1. Por lo tanto, se usa el intervalo con **corrección de Welch**.

Paso 2. Intervalo de Welch: $(\bar{x}_1 - \bar{x}_2) \pm t_{\alpha/2,\, v} \cdot \sqrt{(s_1^2/n_1 + s_2^2/n_2)}$, con v calculado por la aproximación de Welch-Satterthwaite.

```
a, b = v1 / n1, v2 / n2
ee = np.sqrt(a + b)
gl = (a + b) ** 2 / (a ** 2 / (n1 - 1) + b ** 2 / (n2 - 1))   # Welch-Satterthwaite
t  = stats.t.ppf(0.975, df=gl)
LI, LS = (m1 - m2) - t * ee, (m1 - m2) + t * ee
# Validación: stats.ttest_ind(micro, mono, equal_var=False).confidence_interval(0.95)
```

Diferencia (ms)	EE (ms)	v (gl)	t crítico	IC 95 % (ms)	¿0 ∈ IC?
2.6094	0.0309	368.13	1.9664	[2.5487, 2.6701]	**NO**

IC del 95 % para $\mu_{micro} - \mu_{mono}$ (Welch). Los límites coinciden con los calculados por scipy.



El intervalo está muy alejado de la diferencia nula.

7.3 Interpretación de $0 \notin$ IC

- Con 95 % de confianza la diferencia real **no es nula**: los microservicios son más lentos y no por azar.
- La magnitud está acotada con precisión (±0.061 ms): la arquitectura de microservicios añade entre 2.55 y 2.67 ms por petición, aproximadamente **0.87 ms por cada salto HTTP**; en términos relativos, entre +192 % y +201 %, es decir, la latencia casi se triplica.
- Por la dualidad entre intervalos y pruebas, como el intervalo completo es positivo, el contraste del Bloque 4 rechazará H_0.

8. Bloque 4 — Teoría de decisión estadística (prueba de hipótesis)

8.1 Planteamiento de hipótesis

El problema pregunta si la sobrecarga de red **incrementa** la latencia; por ello la hipótesis de investigación es direccional y se usa una prueba de **cola derecha**:

H_0: $\mu_{micro} \leq \mu_{mono}$ (los microservicios NO incrementan la latencia)

H_1: $\mu_{micro} > \mu_{mono}$ (los microservicios SÍ incrementan la latencia)

8.2 Matriz de decisión y costo de los errores

Decisión \ Realidad	H_0 verdadera (no hay penalización)	H_0 falsa (sí hay penalización)
Rechazar H_0	**Error Tipo I (α).** Se descartan los microservicios sin motivo o se invierte en optimizaciones innecesarias (gRPC, service mesh, cachés distribuidas). Se pierden escalabilidad, despliegues aislados y tolerancia a fallos.	Decisión correcta (potencia 1 − β). Se identifica y presupuesta la penalización real.
No rechazar H_0	Decisión correcta (1 − α).	**Error Tipo II (β).** Se migra asumiendo latencia igual; en producción los saltos se acumulan en cadenas de llamadas, se violan los SLA/SLO, se degrada la experiencia de usuario y revertir la arquitectura es muy costoso.

Matriz de decisión con costos operativos.

En este contexto el **Error Tipo II es el más costoso**, porque sus consecuencias aparecen tarde (en producción) y su corrección implica rediseñar el sistema. Por ello interesa una prueba con alta potencia, la cual se garantiza con el tamaño de muestra empleado (sección 8.5).

8.3 Nivel de significancia

Se fija **α = 0.05**: se acepta un 5 % de riesgo de cometer Error Tipo I. El valor crítico de la prueba de cola derecha con v ≈ 368.1 grados de libertad es $t_{crít}$ = 1.6490.

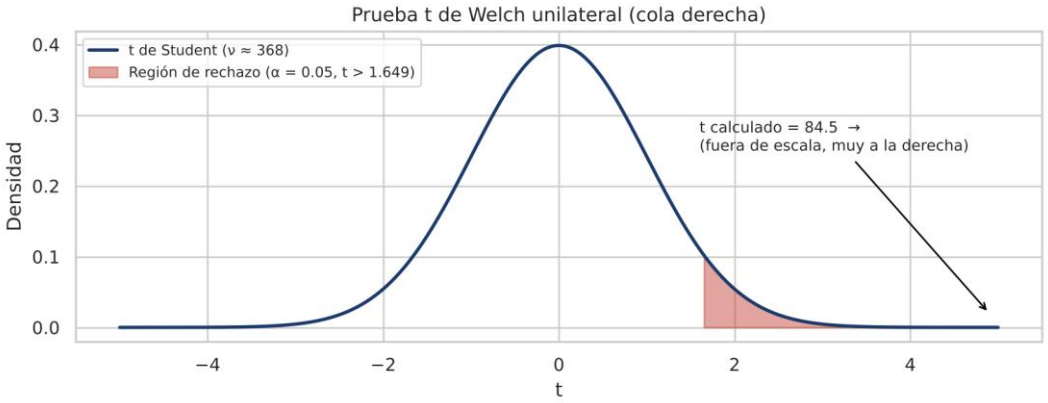
8.4 Ejecución de la prueba paramétrica

Dado que Levene rechazó la igualdad de varianzas, se ejecutó la **prueba t de Welch** con *scipy.stats.ttest_ind*:

```
prueba = stats.ttest_ind(micro, mono, equal_var=False, alternative="greater")
```

Indicador	Valor
Estadístico t (Welch)	84.5249
Grados de libertad v	368.13
p-value (unilateral, cola derecha)	1.75×10^{-243}
p-value (bilateral, referencia)	3.50×10^{-243}
t crítico (α = 0.05, una cola)	1.6490
d de Cohen (tamaño del efecto)	7.02
Mann-Whitney U, p-value (prueba no paramétrica de respaldo)	1.41×10^{-96}

Resultados de la prueba de hipótesis.



Región de rechazo de la prueba unilateral; el t calculado queda fuera de escala.

8.5 Riesgo de Error Tipo II (β) y potencia

Con la distribución t no central se calculó la potencia de la prueba para distintas diferencias de interés práctico:

Δ a detectar (ms)	Potencia (1 − β)	β (riesgo Tipo II)
0.05	0.4887	0.5113
0.10	0.9439	0.0561
0.25	1.0000	0.0000
0.50	1.0000	0.0000
2.61	1.0000	0.0000

Potencia de la prueba t de Welch con los tamaños muestrales del estudio.

La prueba detectaría una penalización de apenas 0.10 ms con probabilidad ≈ 0.94, y cualquier diferencia de 0.25 ms o más con probabilidad prácticamente 1. Solo diferencias minúsculas (≈ 0.05 ms, imperceptibles para el usuario) tendrían riesgo alto de pasar inadvertidas. El riesgo del error más costoso está, por tanto, controlado.

8.6 Veredicto:

Como **$p = 1.75 \times 10^{-243} < \alpha = 0.05$** y $t = 84.52 > t_{crít} = 1.649$, **SE RECHAZA H_0**. Existe evidencia estadísticamente significativa de que la arquitectura de microservicios incrementa la latencia media de respuesta respecto al monolito. El veredicto es robusto: coincide con el IC del Bloque 3 ($0 \notin$ IC), con la prueba no paramétrica de Mann-Whitney ($p \approx 1.41 \times 10^{-96}$) y con un tamaño del efecto muy grande ($d = 7.0$), por lo que no depende del supuesto de normalidad ni de la presencia de sesgo.

9. Recomendación técnica de ingeniería

- **Rutas críticas en latencia** (autenticación, checkout, APIs de tiempo real): mantener la lógica en un monolito o en un solo servicio. La descomposición añade ≈ 2.6 ms por petición (≈ 0.87 ms por salto) **incluso en comunicación local**; en una red real con TLS y balanceadores cada salto cuesta más, de modo que la penalización observada es un **límite inferior**.
- **Si se requieren microservicios** (escalado independiente, equipos autónomos, despliegues aislados), la penalización debe presupuestarse en el SLO y mitigarse: (a) reutilizar conexiones con

keep-alive o connection pooling; (b) paralelizar en el gateway las llamadas independientes (usuarios e inventario no dependen entre sí) con asyncio o hilos; (c) usar protocolos binarios como gRPC/Protocol Buffers en lugar de JSON sobre HTTP/1.1; (d) aplicar Backend for Frontend o caché de agregación para reducir saltos en la ruta crítica.

- **Arquitectura híbrida:** un «monolito modular» para el núcleo transaccional y microservicios solo en los dominios que realmente se benefician del escalado independiente.

10. Limitaciones y trabajo futuro

- Las mediciones se realizaron en una sola máquina, con comunicación por loopback; los valores absolutos no son extrapolables a producción, aunque la dirección del efecto sí lo es.
- La carga fue secuencial (un cliente). Se recomienda repetir el experimento con concurrencia (Locust o Apache Bench) y medir percentiles altos (p95, p99), que suelen ser los más afectados.
- La lógica de negocio es sintética; un estudio posterior podría usar un sistema real e incluir las mitigaciones propuestas para cuantificar cuánta penalización recuperan.

11. Conclusiones

Mediante una prueba de carga controlada se obtuvieron 600 mediciones reales de latencia (579 tras depurar atípicos por IQR). El análisis descriptivo mostró que los microservicios tardan en promedio 3.937 ms frente a 1.328 ms del monolito, con mayor dispersión y sesgo positivo en ambos grupos. La simulación por remuestreo verificó el Teorema Central del Límite incluso sobre una población bimodal, con errores estándar empíricos que coinciden con $s/\sqrt{n}$, lo que respalda el uso de métodos basados en la distribución t.

Al rechazarse la homogeneidad de varianzas, se aplicó la corrección de Welch: el IC del 95 % para la diferencia fue [2.549, 2.670] ms, que no contiene al cero, y la prueba t unilateral rechazó H_0 con $p \approx 1.75 \times 10^{-243}$. Se concluye que la sobrecarga de red de los microservicios incrementa significativamente la latencia, por lo que su adopción debe justificarse por beneficios organizacionales y de escalabilidad, y acompañarse de las mitigaciones descritas para no comprometer los acuerdos de nivel de servicio.

Anexo A. Estructura del repositorio

```
fcd-u2-caso-a/
├── data/
│   ├── dataset_caso_a_raw.csv       # telemetría original (600 registros)
│   └── dataset_caso_a_limpio.csv    # versión depurada por IQR (579 registros)
├── notebooks/
│   └── caso_a_monolito_vs_microservicios.ipynb
├── docs/
│   ├── Reporte_Tecnico_Caso_A.pdf / .docx
│   └── figuras/                     # figuras exportadas por el cuaderno
├── src/
│   └── prueba_carga.py              # arquitecturas + prueba de carga
├── requirements.txt
└── README.md
```